

FIT3155 S1/2026: Assignment 1

(Due date: 11:55pm Friday 1 May 2026)

[Weight: 20 = 10 + 10 marks.]

Assessment guidelines:

- You **must write all code yourself**. Use of Generative AI tools is **not** allowed.
- Any code **derived from online sources must be declared and fully cited**.
- Assessment is about your **understanding and ability to write code yourself**.
- You may be **called for an interview** to justify your understanding of your code.
- You should not use **external library routines**, except those considered standard.
 - Standard data structures (such as lists, dictionaries, tuples, and sets) are allowed if they do not conflict with the assessment objectives. Ensure their use is appropriate and efficient in terms of space and time.

Submission instructions:

- All your scripts **must** contain your name and student ID.
- Upload a **.zip** archive via Moodle with the filename of the form **<student.ID>.zip**.
 - Your archive should extract to a directory named as your Monash student ID.
 - This directory should contain a subdirectory for each of the questions, named as: **q1/** and **q2/**.
 - The corresponding scripts and other question-specific reporting material (as stated below) should be placed in these subdirectories.

Academic integrity

Monash University's academic integrity policy is accessible via this link: <https://www.monash.edu/students/academic/policies/academic-integrity> (click). Read it carefully to understand your responsibilities. **As per FIT policy, all submissions will be scanned via MOSS (click) and/or JPlag (click).**

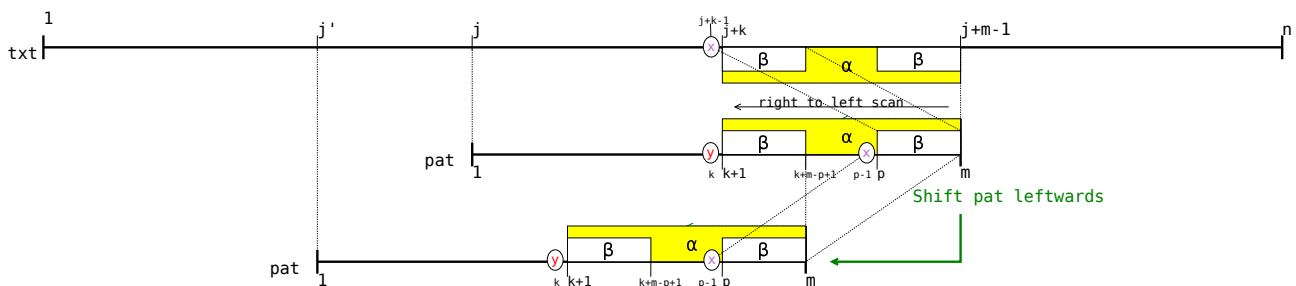
Assignment Questions

Assume all strings used in this assignment are from the alphabet (Σ) containing characters in the ASCII range [37, 126].

1. Yet another exact pattern-matching algorithm:

Here, we describe an algorithm* for finding all occurrences of a pattern $\text{pat}[1 \dots m]$ in a given text $\text{txt}[1 \dots n]$, drawing directly on the concepts covered in weeks 1 and 2.

Overview of this algorithm: This algorithm is designed to shift the pattern **leftwards** relative to the text at the end of each iteration. The process begins with the right ends of both the text and the pattern aligned. In each iteration (as illustrated below), the pattern $\text{pat}[1 \dots m]$ is compared with a substring of the text, $\text{txt}[j \dots j + m - 1]$ (where $1 \leq j \leq n - m + 1$) by carrying out the following steps:



Right-to-left scanning: Scan $\text{pat}[1 \dots m]$ against $\text{txt}[j \dots j + m - 1]$ from right to left until either a mismatch is encountered at position k in the pattern or the entire pattern matches the text region starting at j . Either way, this scan will define a substring $\alpha = \text{txt}[j + k \dots j + m - 1] = \text{pat}[k + 1 \dots m]$, where $0 \leq k \leq m^\dagger$ (see yellow box in the illustration).

Shift pattern leftwards: To go to the next iteration, this algorithm relies on finding the leftmost index $p > k + 1$ (see note below) in the pattern that satisfies the following two conditions:

- $\text{pat}[k + 1 \dots k + m - p + 1] = \text{pat}[p \dots m]$ (substrings β in the illustration),
- $\text{pat}[p - 1] = \text{txt}[j + k - 1]$ (characters x in the illustration).

Note: When $k = m$, clearly $|\alpha| = 0$, which implies $|\beta| = 0$. By definition, $\text{pat}[m + 1 \dots m]$ denotes the empty suffix. In this corner case, consistent with the semantics of p in relation to $|\beta|$, we let p default to $m + 1$.

Finding the leftmost index p satisfying both these conditions will inform the number of positions to shift the pattern **leftwards** safely and efficiently (see illustration above), and carry out its next iteration starting at some $j' < j$. These iterations continue until the pattern can no longer be shifted leftwards relative to the text.

Building on the concepts and ideas introduced in Weeks 1 and 2, your task is to analyze the general iteration described above and work out the details necessary for a space- and time-efficient **0-based** implementation of this pattern-matching algorithm. Carefully read and follow the instructions provided on the next page.

*The algorithm is described here using 1-based indexing. You will have to implement it in 0-based indexing.

[†]When $k = m$, the length of α , $|\alpha| = 0$, a situation that arises when $\text{txt}[j + m - 1] \neq \text{pat}[m]$. On the other extreme, when the whole pattern matches, $|\alpha| = m$.

Strictly follow the following specification to address this question:

Program name: a1q1.py

Arguments to your program: Two filenames:

- (a) First containing `txt[1...n]` string (in a single line of the file).
- (b) Second containing `pat[1..m]` string (in a single line of the file).

(You are free to assume that the text and pattern strings are non-empty and that pattern is at most as long as the text.)

Command line usage of your script:

```
python a1q1.py <text filename> <pattern filename>
```

Do not hard-code the input filenames within your program. Ensure your program can be executed from the terminal (command line) by supplying any pair of filenames (containing text and pattern strings) as command-line arguments. Include sufficient inline comments explaining each logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: output_a1q1.txt

- Each position where `pat` matches the `txt` must be reported on a new line using 1-based indexing (even though your program will compute them in 0-based indexing). If no matches are found, the output file will be empty.

- **Example:** Given `txt = b a b b a a b a b a a b a b a a b a b a b` and
`pat = a a b a b a a b a b`, the output file will contain:
10
5

No further test cases will be provided. You must develop your own test cases.

Runlog: runlog_a1q1.txt

(All indexes in this file should be 0-based, the way your program computes them.)

- Generate a separate runlog file, where each line reports each iteration's variables `<j>` `<k+1>` `<p>` in 0-based indexing, space-separated.
- **Example:** For the example stated above, the runlog file will contain:
11 6 8
9 0 5
4 0 5

No further runlog examples will be provided.

Written PDF Report: report_a1q1.pdf

- Submit a concise report addressing the following technical requirements:
 - Provide a high-level summary of the logic used to identify p for each iteration.
 - Provide a formal or logical proof demonstrating that your implemented shift is safe (i.e., no valid matches are overlooked during shifting).
 - Characterize the worst-case time and space complexity of your implementation with justification.

2. Match-Distance ≤ 1 pattern-matching using BWT as a search index:

Overview: The match-distance between two strings can be defined as the minimum number of edit operations (substitutions, transpositions, insertions, or deletions) required to transform one string into another.

This assignment focuses on identifying the **positions of matches** of `pat[1...m]` within a text `txt[1...n]` using the text's BWT as a search index,[‡] under a distance constraint of ≤ 1 .

Under this constraint, a pattern matches the text starting at position j in the following ways:

- *Match-distance 0 (Exact Match):* A substring of the text is identical to the pattern (e.g., ...FIT3155... vs FIT3155).
- *Match-distance 1 (Substitution):* The text's substring and pattern differ by exactly one character (e.g., ...FIT**9**155... vs FIT3155).
- *Match-distance 1 (Transposition):* The text's substring and pattern become identical upon swapping two **adjacent**, distinct characters (e.g., ...**I**FT3155... vs FIT3155).
- *Match-distance 1 (Insertion):* Adding one character to the pattern makes it identical to the text's substring (e.g., ...FIT3155... vs FIT155[^]).
- *Match-distance 1 (Deletion):* Removing one character from the pattern makes it identical to the text substring (e.g., ...FIT3155... vs FI**S**T3155).[§]

In this task, given any text and a pattern, your first step is to construct the text's BWT. Note that the method of BWT construction will NOT be assessed for efficiency in this assignment. For BWT generation, we expect you to write a straightforward Python function within your code that takes the text string as input, computes its suffix array via *explicit sorting* of all its suffixes (utilizing built-in Python sort methods), and generates a BWT from that computed suffix array. You still have to ensure that you are generating the correct BWT for any given text string, even if the method of BWT generation is not assessed for efficiency. Beyond this, what will be assessed is the correctness and efficiency of your BWT-based search to find all distance ≤ 1 matches of the pattern in the text.

Strictly follow the following specification to address this question:

Program/Function name: `a1q2.py`

Arguments to your program/function: Two filenames, containing:

- (a) the string corresponding to `txt[1...n]` (without any line breaks).
- (b) the string corresponding to `pat[1...m]` (without any line breaks).

(You are free to assume that the text and pattern strings are non-empty and that pattern is at most as long as the text. Separately, your program will have to add the unique terminal \$ character after reading the text string from the file.)

[‡]You should NOT invert the BWT, but rather use BWT as a search index, based on the search-approach covered in Week 3.

[§]Insertion and deletion are symmetric operations; an insertion into the pattern to match the text is equivalent to a deletion from the text to match the pattern. For this assignment, we define these operations based on the modification required for the **pattern** to align with the text.

Command line usage of your script:

```
python a1q2.py <text filename> <pattern filename>
```

Do not hard-code the filenames/input in your function. Ensure that we will be able to run your function from a terminal (command line) supplying any pair of text and pattern filenames as arguments. Give sufficient details in your inline comments for each logical block of your code. Uncommented code or code with vague/sparse/insufficient comments will automatically be flagged for a face-to-face interview with the Chief Examiner before your understanding can be ascertained.

Output file name: output_a1q2.txt

- Report the starting positions of all text substrings that match the pattern with a match-distance ≤ 1 . The output must be sorted by position and adhere to the following rules:
 - All matched positions are reported in **1-based** index.
 - For any given starting position in the text, an exact distance 0 match takes precedence. This means that if a distance 0 match exist, report it (see format below); for the same position, you don't have to report in the output file any distance 1 possibilities.
 - If any starting position in the text yields distance 1 matches under different edit operations (substitution, transposition, etc.), you only report that position once (see format below) and not multiple times.
- Each line of the output reports two numbers (space-separated):
<starting_position_in_txt> <0/1 distance_wrt_pat>

- **Example:**

Given txt = ^{1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20}a b d c a b x d c y a b c d z a b x c d and pat = ^{1 2 3 4}a b c d the output should be:

```
1 1
5 1
10 1
11 0
12 1
16 1
```

Written PDF Report: report_a1q2.pdf

Write a brief report addressing these criteria:

- What is the main idea behind your distance 1 search? (This should be in words; do not reproduce code here.)
- What is the worst-case space and worst-case time complexity of your approach? (Provide a brief justification.)

```
--oOo--
END
--oOo--
```